

2. Restaurant Reservation System

Project Overview

Develop a system where customers can reserve tables and administrators can manage reservations and menus.

Models

User

- UserId: int
- Username: string
- PasswordHash: string
- Email: string
- RegistrationDate: datetime
- PersonId: int
- ICollection<Reservation> Reservations

RoleUser (It should be created using configuration)

Role

- RoleId: int
- RoleName: string (admin, customer)

Person

- PersonId: int
- FirstName: string (20)
- LastName: string (30)
- Phone: string (20)
- Address: string (50)

Note: User – Person has one to one relationship

Reservation

- ReservationId: int
- CustomerId: int (foreign key)
- RestaurantId: int (foreign key)
- StatusId: ReservationStatus
- Date: DateTime
- TableNumber: int
- GuestsCount: int

```
public enum ReservationStatus
{
    Pending = 1,
    Confirmed = 2,
    Canceled = 3
}
```

Menu

- Id: int
- MenuName: string // მაგალითად "Lunch Menu"
- Dishes: ICollection<MenuItem>

// სრული ჯამური თანხის დასათვლელად გირჩევთ დაამატოთ ეს ფროფერტი, რომელიც არის გამოთვლადი და შესაბამისად, დინამიური:

- Decimal TotalPrice => Dishes?.Where(d => d.IsAvailable).Sum(d => d.Price) ?? 0;

MenuItem

- Id: int
- DishName: string
- Price: decimal
- IsAvailable: bool

Restraunt

- RestaurantId: int
- RestaurantName: string
- Location: string (The address or location details)
- Phone: string
- Email: string
- TotalTables: int
- SeatsPerTable: int
- Reservations: ICollection<Reservation>
- Menus: ICollection<Menu>

Repositories & Context class & Registration Services (Program.cs) & appsettings.js

- Determine independently

Service Interfaces

IAdminService

```
User GetUserById(int userId);  
void UpdateUserDetails(int userId, UpdateUserModel model);  
void RemoveUser(int userId);
```

IUserService

- void UserRegistration(UserRegisterModel model)
- User Login(string username, string password);
- User GetProfile();
- void UpdateProfile (UpdateProfileModel model);
- void DeleteProfile (int userId);
- void Logout();

IReservationService

- Reservation MakeReservation(Reservation reservation);
- void CancelReservation(int reservationId);
- IEnumerable<Reservation> GetAllReservations();
- IEnumerable<Reservation> GetReservationsByCustomer(int customerId);
- IEnumerable<Reservation> GetReservationsByDate(DateTime date);
- Reservation GetReservationById(int reservationId);
- void UpdateReservationStatus(int reservationId, ReservationStatus status);
- bool AreSeatsAvailable(int restaurantId, DateTime date, int requestedSeats);

Note: bool AreSeatsAvailable(int restaurantId, DateTime date, int requestedSeats)

იმპლემენტაცია იხ. ქვემოთ:

```
public bool AreSeatsAvailable(int restaurantId, DateTime date, int requestedSeats)  
{  
    var restaurant = _context.Restaurants.FirstOrDefault(r => r.RestaurantId == restaurantId);  
    if (restaurant == null) return false;  
  
    int totalSeats = restaurant.TotalTables * restaurant.SeatsPerTable;  
  
    int reservedSeats = _context.Reservations  
        .Where(r => r.RestaurantId == restaurantId && r.Date.Date == date.Date && r.StatusId !=  
ReservationStatus.Canceled)  
        .Sum(r => r.GuestsCount);
```

```
return (totalSeats - reservedSeats) >= requestedSeats;  
}
```

IMenuService

- MenuItem AddDishToMenu(int menuId, MenuItem item);
- void UpdateDish(int menuItemId, MenuItem item);
- void SetAvailability(int menuItemId, bool isAvailable);
- IEnumerable<Menu> GetMenus();
- IEnumerable<MenuItem> GetDishesByMenu(int menuId);
- MenuItem GetDishById(int dishId);
- IEnumerable<Menu> GetMenusWithAvailableDishes();
- void RemoveDishFromMenu(int menuId, int dishId);

Services

AdminService

Implement IAdminService declared methods

UserService

Implement IUserService declared methods

ReservationService

Implement IReservationService declared methods

MenuService

Implement IMenuService declared methods

Controllers

1. AdminController

AdminService-ის ქვემოთ ჩამოთვლილი მეთოდებისთვის, მოცემულ კონტროლერში მოვახდინოთ შესაბამისი Endpoints-ების და მათთან დაკავშირებული მეთოდების განსაზღვრა/იმპლემენტაცია, სისტემის როლების შესაბამისად:

Admin

- List<Customers> GetAllCustomers();
- User GetUserById(int userId);
- void UpdateUserDetails(int userId, UpdateUserModel model);
- void RemoveUser(int userId);

2. UserController

User Service-ის ქვემოთ ჩამოთვლილი მეთოდებისთვის, მოცემულ კონტროლერში მოვახდინოთ შესაბამისი Endpoints-ების და მათთან დაკავშირებული მეთოდების განსაზღვრა/იმპლემენტაცია, სისტემის როლების შესაბამისად:

Unauthorized

- void UserRegistration(UserRegisterModel model)
- User Login(string username, string password);

Admin, Customer

- User GetProfile();
- void UpdateProfile (UpdateProfileModel model);
- void DeleteProfile (int userId);
- void Logout();

3. ReservationController

- Reservation Service-ის ქვემოთ ჩამოთვლილი მეთოდებისთვის, მოცემულ კონტროლერში მოვახდინოთ შესაბამისი Endpoints-ების და მათთან დაკავშირებული მეთოდების განსაზღვრა/იმპლემენტაცია, სისტემის როლების შესაბამისად:

Customer

- void MakeReservation(Reservation reservation);
- void CancelReservation(int reservationId);

Admin

- List<Reservation> GetAllReservations();
- List<Reservation> GetReservationsByCustomer(int customerId);
- void UpdateReservationStatus(int reservationId, ReservationStatus status);

4. MenuController

- Menu Service-ის ქვემოთ ჩამოთვლილი მეთოდებისთვის, მოცემულ კონტროლერში მოვახდინოთ შესაბამისი Endpoints-ების და მათთან დაკავშირებული მეთოდების განსაზღვრა/იმპლემენტაცია, სისტემის როლების შესაბამისად:

Admin

- void AddDishToMenu(int menuId, MenuItem item);
- void UpdateDish(int menuItemId, MenuItem item);
- void SetAvailability(int menuItemId, bool isAvailable);
- void RemoveDishFromMenu(int menuId, int dishId);

Admin, Customer

- IEnumerable<Menu> GetMenus();
- MenuItem GetDishById(int dishId);
- IEnumerable<Menu> GetMenusWithAvailableDishes();